Numerical Simulation of 2D Granular Media Using the Discrete Element Method

Antoine MILLET–COT Abel Inalegwu ONUH

January 15, 2026

1 Introduction

Granular media, such as sand or grains, exhibit complex mechanical behavior that arises from the collective interactions of large numbers of discrete bodies. Unlike classical fluids or elastic solids, these systems often defy standard continuum descriptions. The Discrete Element Method (DEM) provides a suitable framework to study these dynamics by tracking the motion and collisions of each individual particle. The primary objective of this work is to develop a modular numerical solver in Fortran 90 that is capable of simulating two-dimensional granular flows. The code is designed to efficiently handle neighbor detection, contact force calculation, and time integration for a multi-particle system.

2 Theoretical Framework

The system consists of N spherical particles moving in a two-dimensional plane. The motion of the particles is governed by Newton's laws, subject to external gravitational fields and inter-particle contact forces.

2.1 Forces Acting on a Particle

The net force acting on a given particle i is the sum of gravitational, normal repulsive, and viscous damping forces:

$$\vec{F}_{\text{net}} = \vec{F}_g + \vec{F}_n + \vec{F}_d \tag{1}$$

1. Gravitational Force: Each particle of mass m is subjected to gravity acting in the vertical direction:

$$\vec{F}_g = m\vec{g} \tag{2}$$

2. Normal Repulsive Force: Contact mechanics are modeled using a linear spring-dashpot approach (soft-sphere model). When two particles overlap by a distance $\delta_n > 0$, an elastic restoring force acts along the line of centers:

$$F_{\text{rep}} = k_n \delta_n \tag{3}$$

where k_n is the stiffness coefficient.

3. Viscous Damping Force: To model energy dissipation during inelastic collisions, a damping force is introduced proportional to the overlap velocity $\dot{\delta}_n$:

$$F_{\text{vis}} = c_n \dot{\delta}_n \tag{4}$$

where c_n is the damping coefficient derived to satisfy a specific coefficient of restitution e.

2.2 Analytical Solution (Damped Collision)

To validate the numerical model, we analyze a head-on collision between two identical particles reduced to a single Degree of Freedom (DOF) system. The equation of motion for the overlap $\delta_n(t)$ is a damped harmonic oscillator:

$$m^* \ddot{\delta}_n + c_n \dot{\delta}_n + k_n \delta_n = 0 \tag{5}$$

where $m^* = (1/m_i + 1/m_j)^{-1}$ is the reduced mass. For identical particles, $m^* = m/2$.

The solution for the overlap $\delta_n(t)$ with initial relative velocity v_0 is:

$$\delta_n(t) = \frac{v_0}{\omega_d} e^{-\xi t} \sin(\omega_d t) \tag{6}$$

where $\omega_0 = \sqrt{k_n/m^*}$ is the natural frequency, $\xi = c_n/(2m^*)$ is the decay rate, and $\omega_d = \sqrt{\omega_0^2 - \xi^2}$ is the damped frequency.

The theoretical contact duration t_c (time until δ_n returns to zero) is given by:

$$t_c = \frac{\pi}{\omega_d} \tag{7}$$

Using the physical parameters of our simulation, we can calculate the expected contact duration. Assuming silica grains ($\rho = 2650$ kg/m^3) with an average radius $R \approx 0.6$ mm:

$$t_c^{\text{theory}} = \frac{\pi}{\sqrt{\frac{10^5}{1.19\times10^{-6}}}} \approx 1.08 \times 10^{-5} \text{ s} \tag{8}$$

This theoretical value confirms that our time step of $\Delta t = 10^{-6}$ s is sufficient, providing approximately 10 data points per collision.

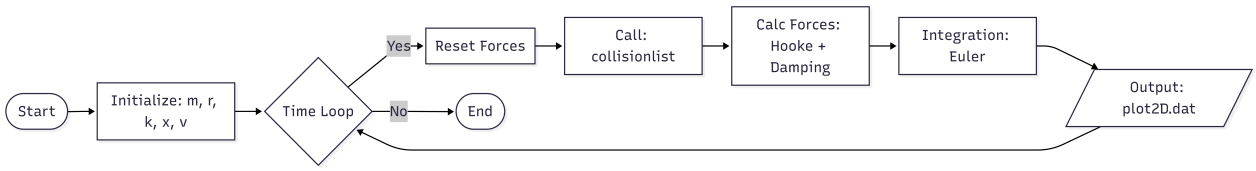


Figure 1: Flowchart of the simulation algorithm executed at each time step.

3 The code

3.1 Files

Fortran and Gnuplot are used to code this simulation. The main code needs vfinw.f90 and collisions.f90 as a module to compile. generateframes.gp is also needed in the same folder.
gfortran collisions.f90 vfinw.f90 -g -o ./main can be used to compile the code. vfinnw.f90 (no walls version) can be used as a replacement for vfinw to see more collisions between the particles as using it removes the damping force at the walls.

3.2 vfinw.f90

This code uses tables of a size matching the number of particles to describe them. Only the values of the current and previous time steps are kept. The bounding box is a box with a corner at 0,0 and another one at xmax,ymax. deltas shouldn't be modified. nframes changes the number of frames in the gif and it doesnt change the simulation. npar changes the amount of particles and if changed, the initial values of some of the tables needs to be changed. There is a main loop updating the tables. In this loop, the projected forces are added and the integration is done. It is necessary to find the particles colliding. A module collisions.f90 returning a list of duos of particles colliding and their distance is used. Then a loop goes through this list and adds the repulsive and damping forces to the two particles. delta and ddelta are the overlap and its derivative. p1 and p2 are the indices of the colliding particles with dist being their distance. Then the collision between each particle and the bounding box is detected and the repulsive and damping forces are added. The difference between vfinw and vfinnw is here: in vfinnw, there is no damping at the walls. At the end of this loop, the integration is done and data is being written into a file to be further processed. The energy is also calculated and written on a file. After this loop, a gp file is created to share useful variables to the post processing. This was added after finding that this was possible quite late into the project as before manually changing the postprocessing file was necessary when changing for example a radius. Then the post processing file making the frames in an automatically created folder is called before finally creating the gif, the output.

3.3 collisions.f90

This modules finds the collisions efficiently by calculating the distance between a particle and the other ones while avoiding to calculate it twice. Then if the distance is lower than the sum of the two radii, a collision is detected, a global table is updated with the index of the two particles and another table is updated with the distance (previously, the index and distance were put in the table and the conflicting types were an issue).

3.4 generateframes.gp

This file is used to create the frames. It uses the parameters given by the file params.gp and the positions of the particles in plot2d.dat to create the frames inside of the frames folder.This file was also changed to allow the use of the parameters and of a varying amount of circles.

3.5 Numerical Stability

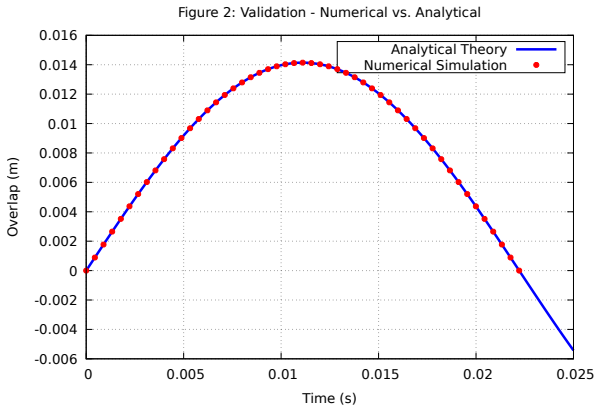
A critical challenge was ensuring the stability condition $\Delta t \ll t_c$. As calculated in Section 2.3, the contact duration is $\approx 10^{-5}$ s. Our chosen time step of 10^{-6} s satisfies this requirement, preventing particle overlap errors and numerical divergence.

4 Results and Discussion

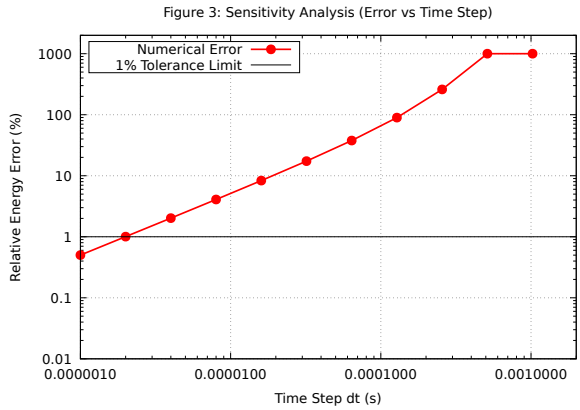
4.1 Solver Validation: Analytical Comparison and Stability

To verify our numerical implementation, we simulated a benchmark head-on collision. Figure 2a presents the comparison between the numerical results (red points) and the exact analytical solution (blue line). The excellent agreement confirms that our force calculation and time-integration schemes are mathematically correct.

We also verified the stability of the explicit Euler scheme. As predicted by the stability condition $\Delta t \ll \sqrt{m/k}$, the simulation remains stable for small time steps. As shown in Figure 2b, the system diverges for $\Delta t > 10^{-4}$ s, confirming the necessity of our chosen $\Delta t = 10^{-6}$ s.



(a) Numerical vs Analytical Solution



(b) Error vs. Time Step

Figure 2: Solver Verification: (a) The numerical integration perfectly reproduces the theoretical sine-wave overlap. (b) Stability analysis showing divergence at large Δt.

4.2 Intermediate Verification: Energy Conservation (No Wall Damping)

Prior to introducing wall damping effects, an intermediate verification was performed using the `vfinnw` code. The objective was to verify that the collision logic was stable and to isolate potential sources of artificial energy drift.

Results showed that particles bounced and exchanged momentum correctly during the initial phase. However, a specific numerical instability was observed regarding mass disparity. When a small particle collided with a significantly larger particle, the velocity transfer occasionally resulted in an excessive velocity gain for the smaller mass. Over time, this led to an artificial increase in the total system energy (numerical blow-up) rather than conservation.

Despite this long-term instability, we retained this code version as a diagnostic tool. It proved valuable for stress-testing the collision detection algorithm under high-frequency impact conditions, which occur more often in the undamped case than in the final damped simulation.

4.3 Final Granular Simulation

Following validation, the final production simulation (`vfinw`) was performed with the full system of 5 grains using the calibrated parameters ($e = 0.7$, $k = 10^5$ N/m).

To visualize the system dynamics, a snapshot of the simulation during the settling phase is shown in Figure 3 illustrates the multi-body interactions within the 0.01×0.01 m domain.

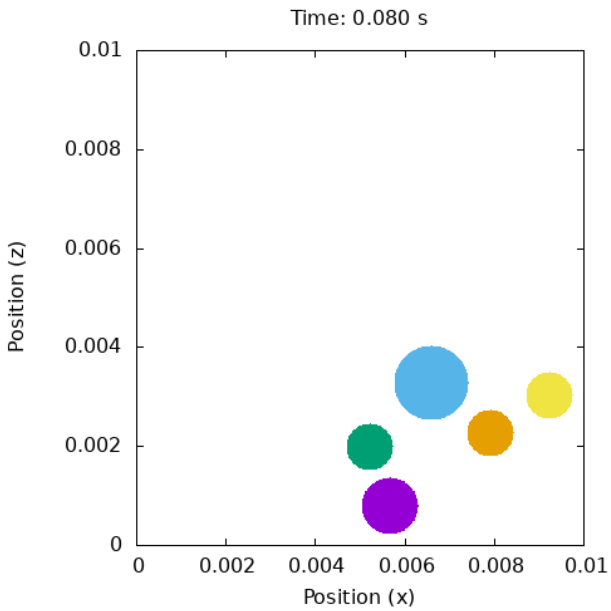


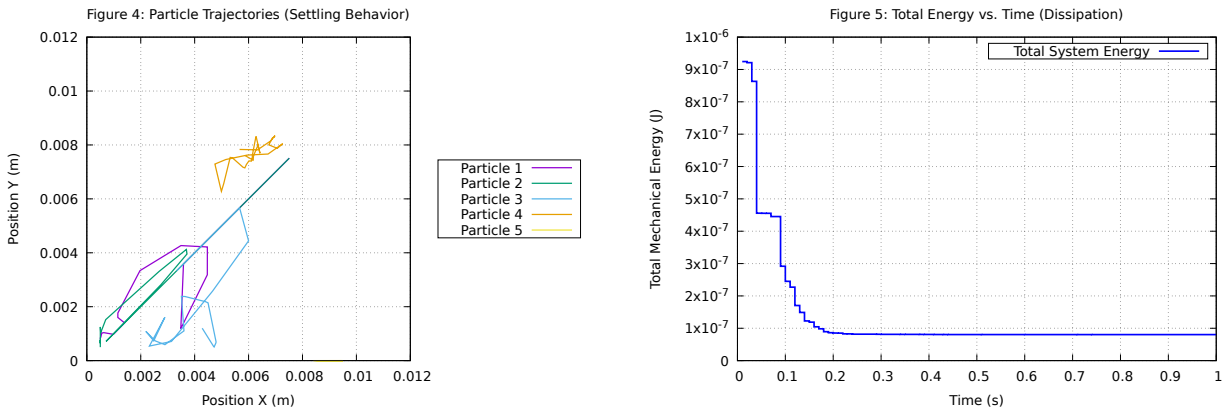
Figure 3: Snapshot of the simulation (Frame 8) as particles fall under gravity.

4.3.1 Trajectory and Energy Analysis

Figure 4 presents the system's quantitative behavior. As shown in Figure 4a, the inclusion of the damping term prevents indefinite bouncing, causing particles to rapidly settle into a stable, static configuration. This contrasts sharply with the undamped case, confirming effective kinetic energy dissipation.

The thermodynamic consistency is verified by the energy evolution in Figure 4b, which displays three distinct regimes:

1. **Free flight:** Energy remains constant between impacts.
2. **Impact events:** where sudden energy drops occur as the damping force $F_d = -c\dot{\delta}$
3. **Equilibrium:** where the total energy converges to a constant minimum representing the gravitational potential energy of the static pile.



(a) Particle Trajectories (Y vs X)

(b) Total Mechanical Energy vs. Time

Figure 4: Final Damped Simulation: (a) Particles falling and settling into a compact pile. (b) The energy decay verifies physical dissipation during inelastic collisions.

4.4 Challenges Encountered and Lessons Learned

Several numerical and implementation challenges were encountered during the development of the solver.

- **Wrong parameter values.** At first particles penetrated the floor or vanished. After some time we found out that the particles were not supposed to be bouncing balls and that the stiffness was way higher with very small overlaps. We changed the timestep and some issues disappeared. We then changed the stiffness, mass and scale and the issue of the normal repulsive force giving the particles too much velocity due to δ and therefore making the particles go too fast even for this timestep disappeared .

- **A coincidence.** when the damping force was first added, we added -c*v to the force given to one of the two particles during a collision and then we wrote the right formula while forgetting that this first formula. So this formula and the right formula were both added to the forces acting on one of these particles. Then we saw that the small particle in our code always had too much force (and energy) added to it after a collision and that at the next collision this energy was given to the other particles. In a broad scale, the system would slow down until this small particle collided with the other particles resulting in a chain reaction increasing the energy. We thought that the issue was because of the damping force not being written correctly and not working for particles of different sizes. While the issue was this added force which was always added to the smallest particle because it was always the one which had this added force during a collision because of the way the code searched for collisions. With this we learned to be careful and look outside the box when trying to solve an error which seems to come from the most logical place.

- **Debugging methodology.** Separating the undamped walls and damped solvers (`vfinnw` and `vfinw`) allowed numerical issues to be isolated efficiently. This staged validation approach along with clever warnings significantly reduced debugging time and increased confidence in the final results.

5 Conclusion

This project successfully implemented a 2D granular solver capable of simulating the settling dynamics of N particles of varying diameters in a 2D system, accounting for gravitational, contact, and damping forces. By benchmarking the code against the analytical solution for a damped harmonic oscillator, we confirmed the mathematical accuracy of our force and integration schemes. The simulation demonstrated both numerical stability at small time steps and physical consistency, as evidenced by the step-wise dissipation of total mechanical energy. In the end, the model captured the essential physics of stiff granular interactions and produced realistic settling behavior verified by thermodynamic laws.

References

1. Bonnefoy, O. (2025). *Numerical Simulation of Particulate Systems.* Course notes, École des Mines de Saint-Étienne.

A Appendix

If some other documents not provided are needed (the code for some figures for example), we can provide them. A link to the source code and additional simulation outputs is provided below:

`https://seafile.emse.fr/d/9d0cbe2e888e4a64b8e7/`